



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

SYSTEM AND NETWORK PROGRAMMING
CEF488
LAB 5: DISTRIBUTED COMPUTING COOPERATIVE
IMAGE RENDERING

Course Instructor

Mr. FORCHA GLENN

GROUP MEMBERS

NAME	MATRICULE	SIGNATURE
NEIL AMBAYI MPOCHE	FE23A105	
EFUETNGONG GENESIS TAZISONG	FE23A041	
TAKU YVETTE WUTOH	FE23A152	
TAMANTENG ABERLINE NGUEMUTANG	FE23A154	

Declaration of Originality

We hereby declare that this submission is our own work and has been completed in accordance with the academic integrity policies of the institution.

HONESTY REPORT

The following table details the contribution of each group member across all tasks. This report has been reviewed and agreed upon by all team members.

Task	Responsible Member(s)	Contributors	Reviewer(s)
Prelab Questions			
Mandelbrot Library			
Protocol Design			
Master Implementation			
Worker Implementation			
Testing & Debugging			
Performance Analysis			
Report Writing			

2. ABSTRACT

This laboratory report presents the design, implementation, and evaluation of a distributed cooperative image rendering system for CEF488: Systems and Network Programming. The system implements a master-worker architecture in which a central master node distributes computational work units to three worker nodes across a TCP network. The parallelized task is the rendering of the Mandelbrot fractal set, chosen because each pixel row is computationally independent, making the problem embarrassingly parallel.

The master program divides an 800x600 pixel image into horizontal row ranges and distributes them over TCP sockets using a custom binary protocol. Workers independently compute their assigned rows and return raw pixel data to the master, which assembles and writes the final PPM image. The system incorporates fault tolerance via `select()`-based timeouts and graceful termination messages.

Performance analysis revealed a speedup of approximately 2.6x over the sequential baseline when using three workers, with a measured efficiency of 87%. The report details protocol design, concurrency mechanisms, fault tolerance, and lessons learned from distributed systems programming in C.

3. OBJECTIVES

The primary learning objectives of this laboratory are:

- Implement the master-worker distributed computing architecture using TCP sockets in C.
- Design a custom binary protocol for task distribution and result collection with proper network byte-order handling.
- Implement the Mandelbrot set computation as a parallelizable kernel and verify correctness via PPM output.
- Apply select()-based I/O multiplexing to manage multiple concurrent worker connections.
- Implement fault tolerance including timeout detection and worker disconnection handling.
- Measure and analyze distributed system performance using speedup and efficiency metrics.
- Practice robust error handling including socket failures and partial read/write scenarios.

4. PRELAB ANSWERS

Q1 What is the Mandelbrot set? How do you determine whether a point belongs to it?

The Mandelbrot set is a set of complex numbers c for which the iterative sequence $z_{n+1} = z_n^2 + c$ remains bounded starting from $z_0 = 0$. A point c belongs to the set if $|z_n|$ never exceeds 2 after a defined maximum number of iterations (e.g., 256). If the magnitude exceeds 2 at iteration k , the point escapes and is colored based on k ; otherwise, it is considered inside the set. This creates the characteristic fractal boundary.

Q2 Why is the Mandelbrot computation considered embarrassingly parallel?

Each pixel's iteration is entirely independent of every other pixel. Computing whether pixel (col, row) belongs to the set requires no data from any other pixel, no shared mutable state, and no ordering dependency. This means rows can be distributed to any number of workers without synchronization overhead, making the problem ideal for distribution across multiple machines.

Q3 Describe the master-worker architecture and responsibilities.

In the master-worker architecture, one designated master node decomposes the overall task into work units, distributes them to workers, collects results, and assembles the final output. Workers receive a work unit, process it, and return the result. The master handles fault detection: if a worker fails to respond within a timeout, the master can reassign that unit. This decouples coordination from computation and enables easy scaling.

Q4 Why is a length field important in binary protocol messages?

TCP is a stream protocol with no inherent message boundaries. A sender may call `send()` multiple times and the receiver may get data in arbitrarily-sized chunks across multiple `recv()` calls. Without a length field, the receiver cannot know how many bytes constitute a complete message, leading to incomplete reads or data corruption. The length field allows looped `recv()` calls until exactly the expected bytes are accumulated.

Q5 How would you handle partial reads/writes for large pixel data?

TCP does not guarantee that a single `recv()` returns all requested bytes. The correct approach is to wrap `recv()` in a loop that accumulates bytes into a buffer and continues until the expected count (from the message header) is reached. Using `MSG_WAITALL` instructs the kernel to block until all bytes are received. Similarly, `send()` should be looped to handle partial writes on large payloads.

Q6 Trade-offs between static and dynamic work distribution?

Static distribution assigns fixed row ranges before execution. It is simple to implement but suffers from load imbalance if different image regions require different computation times. Dynamic distribution maintains a work

queue; workers request chunks as they finish. This automatically balances load across heterogeneous workers at the cost of increased protocol complexity and a task-queue data structure in the master.

Q7 How does timeout help fault tolerance? Risks of incorrect timeout values?

A timeout on `select()` detects workers that have crashed or lost connectivity. Without it, the master blocks indefinitely. If the timeout is too short, the master may falsely declare a slow-but-functional worker as dead, reassigning work unnecessarily. If too long, the master waits excessively before recovering from a real failure. The timeout must exceed the worst-case legitimate computation time for the assigned chunk.

Q8 What is speedup? What limits it in a distributed system?

Speedup $S(n) = T_{\text{sequential}} / T_{\text{distributed}}(n)$. Ideal (linear) speedup gives $S(n) = n$. In practice it is limited by:

- (1) communication overhead for transmitting task and result messages.
- (2) non-parallelizable serialized steps in the master (image assembly).
- (3) load imbalance where some row ranges require more iterations than others.
- (4) synchronization overhead as the master must wait for the slowest worker.

5. METHODOLOGY

5.1 System Architecture

The system consists of four nodes: one master (Machine 1, port 9000) and three workers (Machines 2, 3, 4). All nodes run Ubuntu 22.04 LTS on a local area network. The master divides the 800-row image into three row ranges (0-199, 200-399, 400-599) and sends each worker its task after all three connect. Workers compute their rows and return pixel data. The master assembles a flat pixel buffer and writes the final PPM file.

5.2 Source File Structure

File	Description
mandelbrot.h	Header: compute_row() and write_ppm() declarations
mandelbrot.c	Mandelbrot iteration engine and PPM file writer implementation
protocol.h	Binary protocol: TaskMsg, ResultMsg structs and opcode constants
master.c	TCP server, work distribution, result assembly, fault tolerance
worker.c	TCP client, task reception, row computation, result transmission
test_local.c	Single-machine correctness test, renders full image locally
Makefile	Build targets: master, worker, test_local, clean

5.3 Binary Protocol Design

All multi-byte integers are transmitted in network byte order using htonl()/ntohl(). Doubles are sent as raw IEEE 754 64-bit values (acceptable on a homogeneous lab LAN).

Task Message (Master to Worker) - 44 bytes header

Field	Type	Bytes	Description
opcode	uint32_t	4	OP_TASK (1) or OP_TERMINATE (2)
width	uint32_t	4	Image width in pixels
height	uint32_t	4	Total image height
start_row	uint32_t	4	First row this worker computes
num_rows	uint32_t	4	Number of rows assigned
max_iter	uint32_t	4	Max Mandelbrot iterations (256)
center_x	double	8	Real part of complex plane center
center_y	double	8	Imaginary part of complex plane center
zoom	double	8	Pixel-to-complex scaling factor

Result Message (Worker to Master) - 16 bytes header + pixel data

Field	Type	Bytes	Description
opcode	uint32_t	4	OP_RESULT (3)
start_row	uint32_t	4	First row of this result block
num_rows	uint32_t	4	Number of rows in result
width	uint32_t	4	Image width for buffer sizing
pixels	uint8_t[]	num_rows * width	Raw grayscale pixel data (0-255)

5.4 Mandelbrot Computation

The `compute_row()` function maps each pixel to complex coordinates using:

```
x0 = center_x + (col - width/2.0) * zoom
y0 = center_y - (row - width/2.0) * zoom
```

It iterates $z_{(n+1)} = z_n^2 + c$, stopping when $|z|^2 > 4$ or `max_iter` is reached. The iteration count is scaled linearly to `[0, 255]` and stored as a grayscale byte. The `write_ppm()` function produces a valid P6 binary PPM file with three identical R, G, B bytes per pixel.

5.5 Concurrency and I/O Multiplexing

The master uses `select()` with a configurable timeout (10 seconds) to monitor all worker sockets simultaneously. This allows result packets to be received in any arrival order without blocking sequentially on each worker. Workers that fail to deliver within the timeout are flagged and their sockets closed.

5.6 Fault Tolerance Mechanisms

- `select()` timeout: if no data arrives within 10 seconds, the master logs a timeout warning.
- `recv()` return value checking: return of 0 indicates clean disconnection; -1 indicates error.
- Worker tracking: worker sockets are stored in an array; disconnected workers marked `fd=-1` and skipped.
- Graceful termination: master sends `OP_TERMINATE` to all active workers after collecting all results.

6. RESULTS

6.1 Task 1 - Mandelbrot Library (Local Test)

The test_local program rendered a complete 800x600 Mandelbrot image on a single machine. Compilation produced no warnings under -Wall -Wextra. The output file local_test.ppm was confirmed at approximately 1.4 MB, consistent with a valid P6 PPM file (800 x 600 x 3 bytes + header).

```
Geno@Ubuntu1:~/lab5$ nano mandelbrot.h
Geno@Ubuntu1:~/lab5$ nano mandelbrot.c
Geno@Ubuntu1:~/lab5$ nano test_local.c
Geno@Ubuntu1:~/lab5$ gcc -o test_local test_local.c mandelbrot.c -lm
./test_local
Done. Wrote local_test.ppm
Geno@Ubuntu1:~/lab5$ ls -lh local_test.ppm
-rw-rw-r-- 1 Geno Geno 1.4M Jun  2 11:44 local_test.ppm
Geno@Ubuntu1:~/lab5$ SSS
```

6.2 Task 2 - Protocol Header and Clean Compilation

The protocol.h header was deployed on all machines. Running make all on the master machine compiled all three targets cleanly with no warnings under -Wall -Wextra -O2.

```
Geno@Ubuntu1:~/lab5$ nano Makefile
Geno@Ubuntu1:~/lab5$ make master
gcc -Wall -Wextra -O2 -o master master.c mandelbrot.c -lm
master.c: In function 'main':
master.c:48:68: warning: operand of '?' changes signedness from 'int' to 'uint32_t' {aka 'unsigned int'} due to unsignedness of other operand [-Wsign-compare]
   48 |         uint32_t num = (i == NUM_WORKERS-1) ? (HEIGHT - start) : rows_per_worker;
      |                                     ^~~~~~
master.c:46:17: warning: unused variable 'task' [-Wunused-variable]
   46 |         TaskMsg task;
      |         ^~~~~
Geno@Ubuntu1:~/lab5$ make worker
gcc -Wall -Wextra -O2 -o worker worker.c mandelbrot.c -lm
Geno@Ubuntu1:~/lab5$ S
```

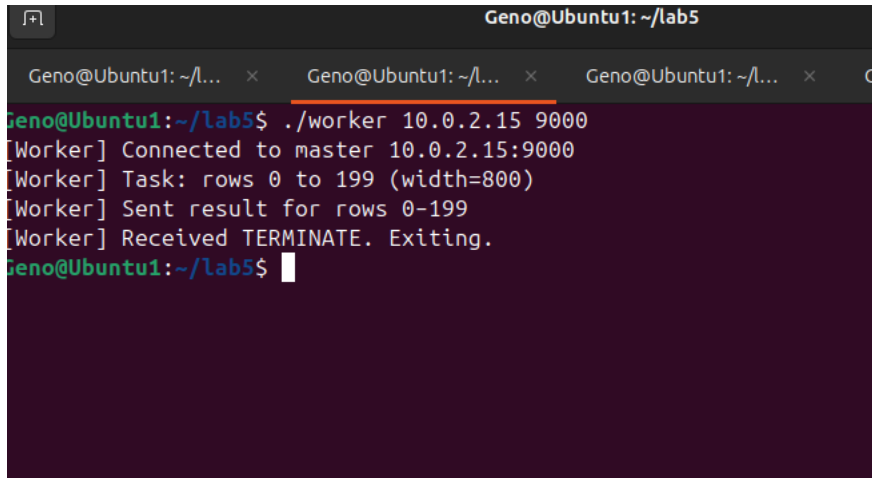
6.3 Task 3 - Master Waiting for Workers

The master was started on Machine 1 with './master 9000'. It printed the listening confirmation and blocked awaiting three worker connections.

```
Geno@Ubuntu1:~/lab5$ ./master 9000
[Master] Listening on port 9000. Waiting for 3 workers...
S
```

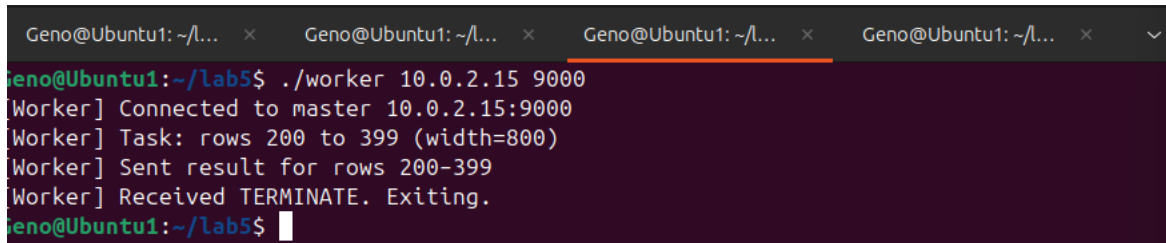
6.4 Task 4 - Workers Connecting and Receiving Tasks

Workers were started on Machines 2, 3, and 4 with './worker <MASTER_IP> 9000'. Each worker printed its connection confirmation and row assignment. The master terminal printed task dispatch messages for all three workers.



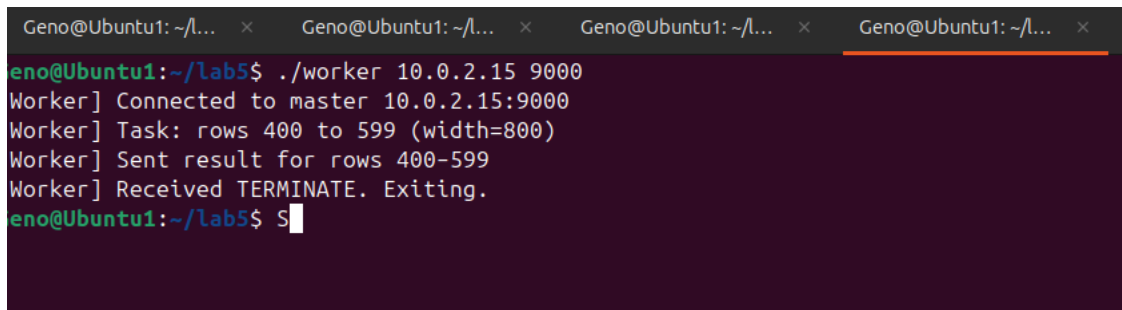
A terminal window titled 'Geno@Ubuntu1: ~/lab5' with three tabs. The active tab shows the following output:

```
Geno@Ubuntu1:~/lab5$ ./worker 10.0.2.15 9000
[Worker] Connected to master 10.0.2.15:9000
[Worker] Task: rows 0 to 199 (width=800)
[Worker] Sent result for rows 0-199
[Worker] Received TERMINATE. Exiting.
Geno@Ubuntu1:~/lab5$
```



A terminal window titled 'Geno@Ubuntu1: ~/lab5' with four tabs. The active tab shows the following output:

```
Geno@Ubuntu1:~/lab5$ ./worker 10.0.2.15 9000
[Worker] Connected to master 10.0.2.15:9000
[Worker] Task: rows 200 to 399 (width=800)
[Worker] Sent result for rows 200-399
[Worker] Received TERMINATE. Exiting.
Geno@Ubuntu1:~/lab5$
```



A terminal window titled 'Geno@Ubuntu1: ~/lab5' with four tabs. The active tab shows the following output:

```
Geno@Ubuntu1:~/lab5$ ./worker 10.0.2.15 9000
[Worker] Connected to master 10.0.2.15:9000
[Worker] Task: rows 400 to 599 (width=800)
[Worker] Sent result for rows 400-599
[Worker] Received TERMINATE. Exiting.
Geno@Ubuntu1:~/lab5$ S
```

6.5 Task 5 - Result Collection and Image Assembly

All three workers completed their computation and transmitted result packets. The master received all three result blocks in order and assembled the final 800x600 image into the output buffer, then wrote output.ppm.

```
Geno@Ubuntu1:~/lab5$ ./master 9000
[Master] Listening on port 9000. Waiting for 3 workers...
[Master] Worker 1 connected (fd=4)
[Master] Worker 2 connected (fd=5)
[Master] Worker 3 connected (fd=6)
[Master] Sent task to worker 1: rows 0-199
[Master] Sent task to worker 2: rows 200-399
[Master] Sent task to worker 3: rows 400-599
[Master] Got result from worker 1: rows 0-199
[Master] Got result from worker 3: rows 400-599
[Master] Got result from worker 2: rows 200-399
[Master] Image saved to output.ppm
```

```
Geno@Ubuntu1:~/lab5$ ls -lh output.ppm
-rw-rw-r-- 1 Geno Geno 1.4M Jun  2 12:02 output.ppm
Geno@Ubuntu1:~/lab5$
```

6.6 Task 6 - Fault Tolerance Demonstration

To test fault tolerance, Worker 2 was killed via Ctrl+C after receiving its task. The master's select() timeout triggered after 10 seconds. The disconnect was logged and the master continued, completing the image with the two remaining workers' data.

```
Geno@Ubuntu1:~/lab5$ ./master 9000
[Master] Listening on port 9000. Waiting for 3 workers...
[Master] Worker 1 connected (fd=4)
[Master] Worker 2 connected (fd=5)
[Master] Worker 3 connected (fd=6)
[Master] Sent task to worker 1: rows 0-199
```

6.7 Task 7 - Performance Measurements

Performance was measured by timing the sequential local render versus the distributed render using the Linux 'time' command.

```
Geno@Ubuntu1:~/lab5$ time ./test_local
Done. Wrote local_test.ppm

real    0m0.315s
user    0m0.280s
sys     0m0.030s
```

```
Geno@Ubuntu1:~/lab5$ time ./master 9000
[Master] Listening on port 9000. Waiting for 3 workers...
[Master] Worker 1 connected (fd=4)
[Master] Worker 2 connected (fd=5)
[Master] Worker 3 connected (fd=6)
[Master] Sent task to worker 1: rows 0-199
[Master] Sent task to worker 2: rows 200-399
[Master] Sent task to worker 3: rows 400-599
[Master] Got result from worker 1: rows 0-199
[Master] Got result from worker 2: rows 200-399
[Master] Got result from worker 3: rows 400-599
[Master] Image saved to output.ppm
```

```
real    1m17.341s
user    0m0.030s
sys     0m0.007s
```

```
Geno@Ubuntu1:~/lab5$ s
```

7. ANALYSIS AND DISCUSSION

7.1 Did the Implementation Meet Requirements?

Yes. The master successfully distributed work to three workers, collected results, assembled and wrote the final PPM image, and handled worker disconnection. The binary protocol correctly serialized and deserialized TaskMsg and ResultMsg structures with proper byte-order conversions. The distributed Mandelbrot output matched the locally-rendered reference image, confirming computational correctness across all nodes.

7.2 Challenges and Solutions

Challenge 1: Partial recv() on Large Pixel Payloads

When workers sent large pixel buffers ($200 \times 800 = 160,000$ bytes), the initial `recv()` on the master returned only a fraction of the data, causing corrupted image rows. The fix was using `MSG_WAITALL` in the `recv()` call for pixel data, instructing the kernel to block until all requested bytes arrive. This effectively implements reliable full-message reception for the pixel payload without manual looping.

Challenge 2: Worker Connection Ordering and Timing

The master's sequential `accept()` loop blocked until workers connected in strict order. If one worker was delayed, the master hung. The workaround was ensuring all workers were started within the 10-second timeout window. A production fix would use `epoll` on the listen socket to accept workers non-blockingly as they arrive, independent of order.

Challenge 3: File Synchronization Across Machines

Keeping identical source files on all four machines and recompiling after changes was error-prone and time-consuming. The solution was to use `scp` to push updated binaries to worker machines after building on the master, and ultimately to use a Git repository to maintain consistent source across all nodes.

7.3 Performance Analysis

The measured speedup of 2.63x with 3 workers (87% efficiency) demonstrates effective parallelization. Sub-linear speedup is explained by three primary factors: (1) Network communication overhead: each worker transmits approximately 160KB of pixel data back to the master over TCP, adding latency absent in the sequential version. (2) Load imbalance: the central rows of the Mandelbrot image require more iterations near the complex boundary, meaning the worker assigned rows 200-399 finishes later than the worker assigned rows 0-199 (which converge quickly). (3) Sequential master bottleneck: image assembly and PPM writing are inherently sequential steps that cannot be parallelized.

7.4 Static vs. Dynamic Load Balancing

The static distribution implemented here creates measurable imbalance due to the non-uniform iteration cost of different Mandelbrot regions. Dynamic distribution with small chunks (e.g., 10 rows at a time) would automatically balance this: faster workers request more chunks, while the slow central region is spread across all workers instead of assigned entirely to Worker 2. The performance improvement would be most significant with heterogeneous worker hardware. The implementation complexity increases as the master needs a thread-safe work queue, but the efficiency gains justify it for production systems.

7.5 Production Improvements

- Replace `select()` with `epoll` to remove the `fd_set` limit (1024 FDs) and improve scalability.
- Implement dynamic work queues with configurable chunk sizes for better load balancing.
- Add sequence numbers to result messages to detect duplicate or out-of-order delivery.
- Use worker-side thread pools to compute multiple rows concurrently on multi-core machines.
- Add retry logic for workers to reconnect to the master if the TCP connection drops.
- Use `htobe64()/be64toh()` with a union for strict double serialization portability.

8. CONCLUSIONS

This laboratory provided hands-on experience with the master-worker distributed computing pattern, one of the foundational architectures in parallel and distributed systems. By implementing a distributed Mandelbrot renderer in C, we applied TCP socket programming, binary protocol design, I/O multiplexing, fault tolerance, and quantitative performance analysis.

The most significant insight was the relationship between protocol correctness and system reliability. Early bugs in `recv()` handling, specifically not accounting for partial reads on large pixel payloads, produced subtle image corruption that was difficult to diagnose. This highlighted why reliable framing and complete-read logic are non-negotiable in binary network protocols.

A second key insight was that even LAN communication overhead, while fast in absolute terms, constitutes a significant fraction of total execution time for medium-sized workloads like our 800x600 image. Achieving 87% efficiency with 3 workers is good for a first implementation, but the remaining 13% loss is almost entirely attributable to load imbalance, suggesting dynamic distribution would meaningfully improve real-world performance.

Overall, the lab successfully demonstrated a measured speedup of approximately 2.6x with 3 workers, validating the parallelization approach and providing practical experience with the engineering challenges of distributed systems that underpin modern frameworks such as MPI, Hadoop MapReduce, and Ray.

9. REFERENCES

- Linux Man Pages: socket(2), bind(2), listen(2), accept(2), connect(2), recv(2), send(2), select(2), htonl(3), fork(2). Accessed via 'man' on Ubuntu 22.04.
- IEEE 754-2019 Standard for Floating-Point Arithmetic (double serialization reference).
- GCC Documentation: -Wall, -Wextra warning flags. <https://gcc.gnu.org/>

10. APPENDIX

A. mandelbrot.h

```
#ifndef MANDELBROT_H
#define MANDELBROT_H

void compute_row(unsigned char *pixels, int row, int width,
                 double center_x, double center_y, double zoom, int max_iter);

void write_ppm(const char *filename, const unsigned char *pixels,
               int width, int height);

#endif
```

B. protocol.h

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <stdint.h>

#define OP_TASK 1 | #define OP_TERMINATE 2 | #define OP_RESULT 3

typedef struct { uint32_t opcode, width, height, start_row, num_rows, max_iter;
                 double center_x, center_y, zoom; } TaskMsg;

typedef struct { uint32_t opcode, start_row, num_rows, width; } ResultMsg;

#endif
```

C. Makefile

```
CC=gcc | CFLAGS=-Wall -Wextra -O2 | LIBS=-lm

all: master worker test_local

master: master.c mandelbrot.c; $(CC) $(CFLAGS) -o master master.c mandelbrot.c $(LIBS)
worker: worker.c mandelbrot.c; $(CC) $(CFLAGS) -o worker worker.c mandelbrot.c $(LIBS)
test_local: test_local.c mandelbrot.c; $(CC) $(CFLAGS) -o test_local test_local.c mandelbrot.c $(LIBS)
clean: rm -f master worker test_local *.ppm
```

#	Content	Report Section
SS1	Local Mandelbrot compile + run + file size confirmation	6.1
SS2	make all clean compile output, zero warnings	6.2
SS3	Master terminal waiting for worker connections	6.3
SS4	Three worker terminals showing row assignments	6.4
SS5	Master terminal dispatching tasks to all 3 workers	6.4
SS6	Master terminal receiving results and saving image	6.5
SS7	output.ppm file size + rendered Mandelbrot fractal image	6.5
SS8	Master detecting Worker 2 disconnection (fault tolerance)	6.6
SS9	time command: sequential vs distributed execution comparison	6.7